



第八章 查找

杨震 计算机学院

8.1 基本概念与术语

BUPT

查找表 同一类型的记录(数据元素)的**集合**。

查找 指定某个值，在查找表中确定是否存在一个记录，该记录的**关键字**等于给定值。

关键字 记录(数据元素)中的某个数据项的值。

主关键字 该关键字可以唯一地标识一个记录。

次关键字 该关键字不能唯一标识一个记录。

静态查找表 对查找表的查找仅是以查询为目的，不改动查找表中的数据。

动态查找表 在查找的过程中同时插入不存在的记录，或删除某个已存在的记录。

查找成功 查找表中存在满足查找条件的记录。

查找不成功 查找表中不存在满足查找条件的记录。

内查找 整个查找过程都在内存中进行。

外查找 在查找过程中需要访问外存。

平均查找长度ASL——查找方法时效的度量

为确定记录在查找表中的位置，需将关键字和给定值比较次数的期望值。

查找成功时的ASL计算方法：
$$ASL = \sum_{i=1}^n p_i c_i$$

n : 记录的个数

p_i : 查找第 i 个记录的概率，

(不特别声明时认为等概率 $p_i = 1/n$)

c_i : 找到第 i 个记录所需的比较次数

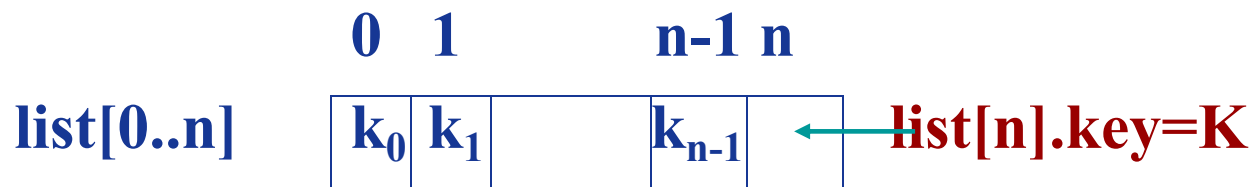
约定：无特殊说明，一般默认关键字的类型为整型

本章中如未做特殊声明，将采用如下顺序存储结构。

```
#define MAXSIZE 1000  
typedef struct  
{  
    int key;  
    /* otherinfo field;*/  
} elemtype;  
elemtype list[MAXSIZE];
```

8.2 顺序表的查找 快插入慢查找

BUPT



[算法描述]

```
int seqsearch (elemtype list[], const int n, const int K )
{
    int i = 0;
    list[n].key = K; //设置监视哨，简化操作
    while (list[i].key != K)
        i++;
    return ((i < n) ? i - 1); //查找成功返回i (0 ≤ i < n), 查找失败返回-1
}
```

BUPT SCST

[程序设计技巧] 设置监视哨，提高算法效率。

[性能分析]

❖ 空间：一个辅助空间。

❖ 时间：

1) 查找成功时的平均查找长度

设表中各记录查找概率相等

$$ASL_s(n) = (1+2+...+n)/n = (n+1)/2$$

2) 查找不成功时的平均查找长度 $ASL_f = n+1$

[算法特点]

❖ 算法简单，对表结构无任何要求 快插入慢查找

❖ n 很大 $\Rightarrow$ 查找效率较低

❖ 改进措施：非等概率查找时，可将查找概率高的记录尽量排在表前部。

8.3 二分查找 慢插入快查找

BUPT

满足 $\text{list}[i].\text{key} \leq \text{list}[i+1].\text{key}$, $0 \leq i < n-1$

仍可用顺序查找，但在找不到时不需比较到表尾，只需比较到比给定值大的记录就可终止。

[折半(二分)查找法]

0	1	2	3	4	5	6	7	8	9	10
05	13	19	21	37	56	64	75	80	88	92
↑low					↑mid					↑high

$$= \lfloor (\text{low} + \text{high}) / 2 \rfloor$$

K=21

l	h	m
0	10	5
0	4	2
3	4	3

查找成功

K=85

l	h	m
0	10	5
6	10	8
9	10	9
9	8	

查找失败

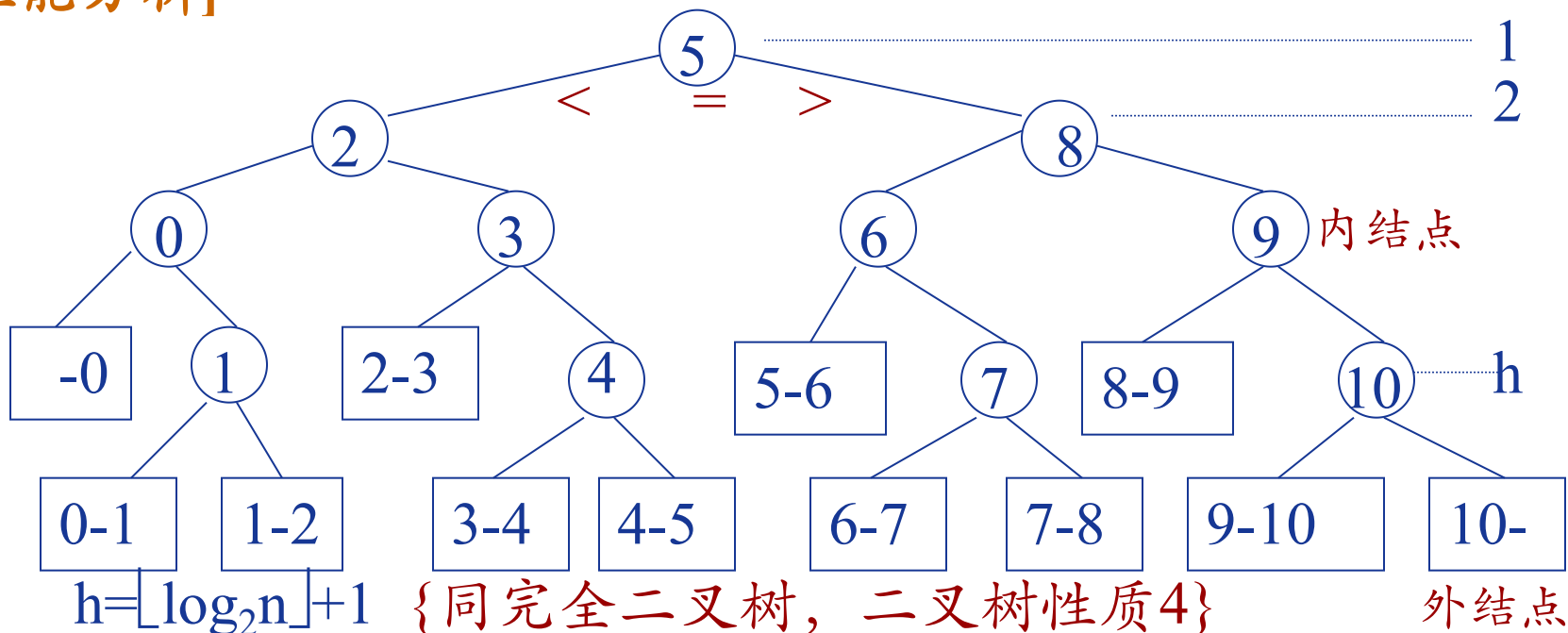
[算法描述]

```
int binsearch (const elemtype list[], const int n, const int K )
```

```
{  
    int low, high, mid;  
    low = 0;  
    high = n-1;  
    while ( low <= high )  
    {  
        mid = ( low + high ) / 2;  
        if ( K < list[mid].key )  
            high = mid - 1;  
        else if ( K > list [mid].key )  
            low = mid + 1;  
        else /* 找到了匹配的值*/  
            return mid; //返回匹配值的数组下标  
    }  
    return -1; /* 没有查到*/  
}
```

[性能分析]

判定树（描述查找过程的二叉树）



成功查找时的平均查找长度（等概率）：

$$ASL_s = \frac{1}{n} \sum_{j=1}^h j \cdot 2^{j-1} = \frac{n+1}{n} \log_2(n+1) - 1 \approx \log_2(n+1) - 1$$

例： $ASL_s = (1*1 + 2*2 + 3*4 + 4*4) / 11 = 3$

不成功查找时的查找长度： $h-1$ 或 h 次

8.4 静态树表的查找

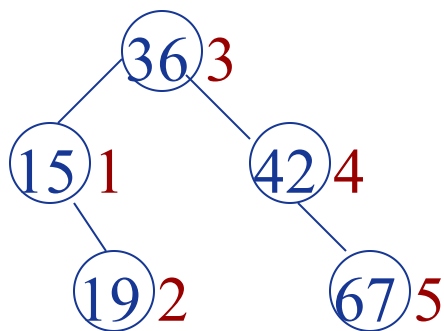
BUPT

[问题出发点]

讨论有序表中各记录查找概率不等时，查找成功时的 ASL_s 。

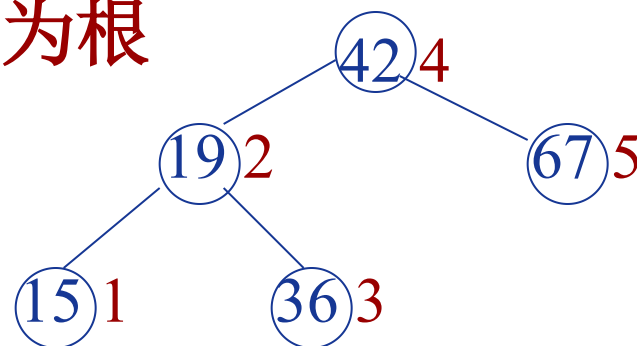
	1	2	3	4	5
关键字	15	19	36	42	67
查找概率	0.1	0.2	0.1	0.4	0.2

折半查找



$$ASL_s = 2.3$$

高概率为根
+折半



$$ASL_s = 1.8$$

[静态最优查找树]

定义：查找性能最佳的判定树。

性质：带权内路径长度之和PH为最小值。

$$PH = \sum_{i=1}^n w_i h_i \quad \text{与} ASL_s \text{成正比}$$

其中n：二叉树上结点的个数(有序表长度)

h_i ：第i个结点在二叉树上的层次数

$w_i = c p_i$ ：c为某个常量；

p_i ：第i个结点的查找概率

最优查找体现的原则：

- 1) 最先访问的结点应是访问概率最大的结点；
- 2) 每次访问应使结点两边尚未访问的结点的被访概率之和尽可能相等。

[次优查找树的构造方法]

- PH值近似为最小
- 比静态最优查找树易于构造，时间开销少

已知按关键字有(升)序的记录序列：

($r_1, r_{l+1}, \dots, r_{i-1}, \mathbf{r_i}, r_{i+1}, \dots, r_{h-1}, r_h$)

1)选取记录 r_i 作为根结点：

计算所有 Δp_i ($1 \leq i \leq h$)
$$\Delta p_i = \left| \sum_{j=i+1}^h w_j - \sum_{j=l}^{i-1} w_j \right|$$

找出最小 Δp_i 的对应的 i ；

若相邻关键字的权值大，则确定为权值大的关键字对应的 i

简化计算 Δp_i 的方法:

设定累计权值和 $sw_i = \sum_{j=0}^i w_j$

$$\text{则 } \sum_{j=l}^{i-1} w_j = sw_{i-1} - sw_{l-1} \quad \sum_{j=i+1}^h w_j = sw_h - sw_i$$

$$\begin{aligned} \Delta p_i &= |(sw_h - sw_i) - (sw_{i-1} - sw_{l-1})| \\ &= |(sw_h + sw_{l-1}) - sw_i - sw_{i-1}| \end{aligned}$$

设 $sw_{-1} = 0$

2) r_i 的左子树: $(r_l, r_{l+1}, \dots, r_{i-1})$ 构造的次优查找树

3) r_i 的右子树: $(r_{i+1}, \dots, r_{h-1}, r_h)$ 构造的次优查找树

设有一组数据black,blue,green,purple,red,white,yellow，它们的查找概率分别为0.10,0.08,0.12,0.05,0.20,0.25,0.20。试以它们的查找概率为权值，构造一棵次优查找树，并计算其查找成功的平均查找长度。

关键字	black	blue	green	purple	red	white	yellow
权值	0.10	0.08	0.12	0.05	0.20	0.25	0.20
j	1	2	3	4	5	6	7
SWj	0.1	0.18	0.30	0.35	0.55	0.80	1.00
ΔP_j	0.9	0.72	0.52	0.35	0.10	0.35	0.80
(根)					$\uparrow i$		
ΔP_j	0.25	0.07	0.13	0.30		0.20	0.25
		$\uparrow i$				$\uparrow i$	
ΔP_j			0.05	0.12			
			$\uparrow i$				

8.5 分块查找

BUPT

[索引表的构造]

索引表 $\text{index}[0..b-1]$

最大关键字值

起始地址

22	42	86
0	4	8

主表 $r[0..n-1]$ (分块有序/有序)

key
其它项

12	22	13	8	28	33	38	42	86	76	50	63
0	1	2	3	4	5	6	7	8	9	10	11

[分块查找步骤]

- 1) 折半或者顺序查找索引表，确定所在块
- 2) 在已确定的块中顺序查找/折半查找

设 b 为索引表长度， s 为块中记录个数

❖ 顺序查找索引表+顺序查找被确定的块

$$ASL_{bs} = \frac{1}{b} \sum_{j=1}^b j + \frac{1}{s} \sum_{i=1}^s i = \frac{1}{2}(b+s) + 1 = \frac{1}{2}(\frac{n}{s} + s) + 1$$

当 s 取 $\sqrt{n}$ 时， ASL_{bs} 取最小值 $\sqrt{n} + 1$

❖ 折半查找索引表+顺序查找被确定的块

$$ASL_{bs} = [\log_2(b+1) - 1] + (s+1)/2 \approx \log_2(n/s+1) + s/2$$

❖ 实用中，各块大小不一定相等

❖ 分块查找效率介于顺序查找和折半查找之间

8.6 二叉排序树BST(二叉查找/搜索树)

BUPT

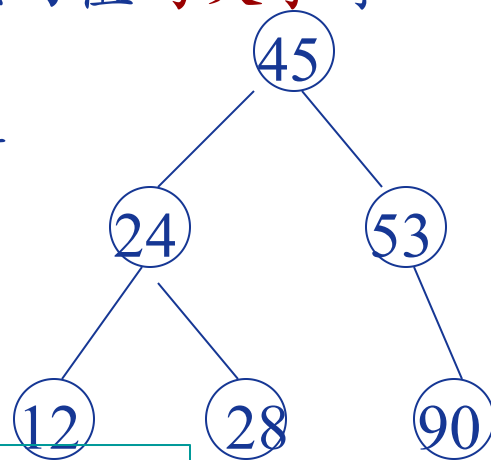
[定义]

二叉排序树或者是空树，或者是满足如下性质的二叉树：

- 1)若其左子树非空，则左子树上所有结点的值均小于根结点的值；
- 2)若其右子树非空，则右子树上所有结点的值均大于等于根结点的值；
- 3)其左右子树本身又各是一棵二叉排序树

[性质]

中序遍历一棵二叉排序树，将得到一个以关键字递增排列的有序序列



求解方法归纳：

- ①一棵BST的最大结点是根结点最右下的结点，即：中序最后一个结点；
- ②一棵BST的最小结点是根结点最左下的结点，即：中序第一个结点；
- ③只需给出BST的先序/后序/层序之一，便可唯一确定这棵BST。

思考题：设计算法求一个给定序列A是否是一个BST的先序序列？

BUPT SCST

[二叉排序树结点声明]

```
struct TreeNode
{
    elemtype data;
    struct TreeNode *left;//左孩子
    struct TreeNode *right;//右孩子
};
typedef struct TreeNode *Position;
typedef struct TreeNode * SearchTree;
```

[函数声明]

SearchTree MakeEmpty(Search T) //建立一颗空的二叉排序树或树置空操作
Position Find(elemtype X, SearchTree T) //在树T上查找值为X的结点并返回
Position FindMin(SearchTree T) //在树T上查找值最小的结点
Position FindMax(SearchTree T) //在树T上查找值最大的结点
SearchTree Insert(elemtype X, SearchTree T) 在树T上插入值为X的结点并返回新树
SearchTree Delete(elemtype X, SearchTree T) 在树T上删除值为X的结点并返回新树

[建空树/树置空]

若树不空则置空返回空，若树为空直接返回空

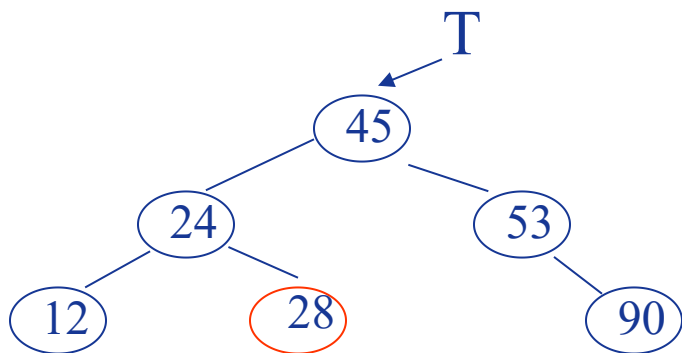
SearchTree MakeEmpty(Search T)

```
{  
    if(T != NULL)  
    {  
        MakeEmpty(T->left);  
        MakeEmpty(T->right);  
        free(T);  
    }  
    return NULL;  
}
```

[查找]

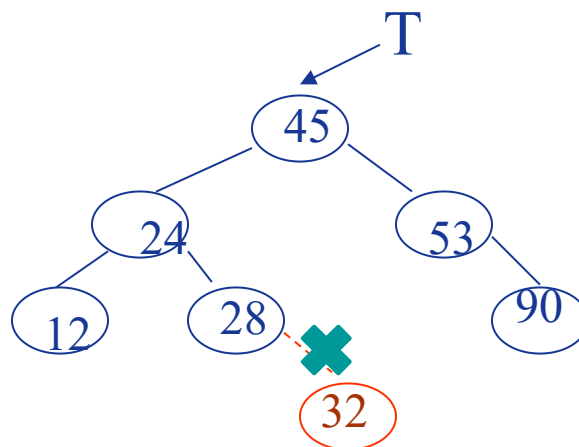
[例]

K=28



查找成功

K=32



查找失败

查找步骤:

若根结点的关键字值等于查找的关键字, 成功。

否则, 若小于根结点的关键字值, 查其左子树

若大于根结点的关键字值, 查其右子树。

在左右子树上的操作类似。

[查找算法]

递归实现

```
Position Find(elemtype X, SearchTree T)
{
    if (T == NULL)
        return NULL;
    if (X < T->data)
        return Find(X, T->left);
    else if (X > T->data)
        return Find(X, T->right);
    else
        return T;
}
```

递归算法性能

时间复杂度: $O(n)$

空间复杂度: 辅助变量是工作栈 $O(h)$

最好情况 $O(\log_2 n)$

最坏情况 $O(n)$

非递归实现

```
Position Find(elemtype X, SearchTree T)
{
    while (T != NULL && T->data != X)
    {
        if (X < T->data)
            T = T->left;
        else
            T = T->right;
    }
    return T;
}
```

非递归算法性能

时间复杂度: $O(n)$

空间复杂度: $O(1)$

[查找最小值/最大值算法]

查找最小值-递归算法

```
Position FindMin(SearchTree T)
{
    if (T == NULL)
        return NULL;
    else if (T->left == NULL)
        return T;
    else
        return FindMin(T->left);
}
```

查找最小值-非递归算法

```
Position FindMin(SearchTree T)
{
    if (T != NULL)
        while (T->left != NULL)
            T = T->left;
    return T;
}
```

查找最大值-递归算法

```
Position FindMax(SearchTree T)
{
    if (T == NULL)
        return NULL;
    else if (T->right == NULL)
        return T;
    else
        return FindMax(T->right);
}
```

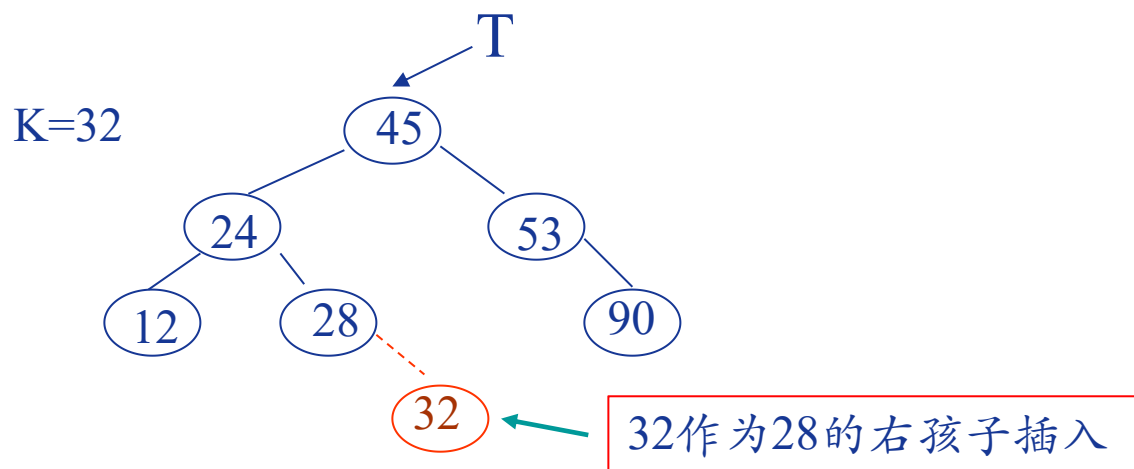
查找最大值-非递归算法

```
Position FindMax(SearchTree T)
{
    if (T != NULL)
        while (T->right != NULL)
            T = T->right;
    return T;
}
```

[插入]

- 首先执行查找算法，找出被插结点的父亲结点。
- 判断被插结点是其父亲结点的左、右儿子。将被
- 插结点作为叶子结点插入。
- 若二叉树为空。则首先单独生成根结点。

注意：新插入的结点总是叶子结点。



[插入算法]

SearchTree Insert(elemtype X, SearchTree T) //返回指向新树根结点的指针

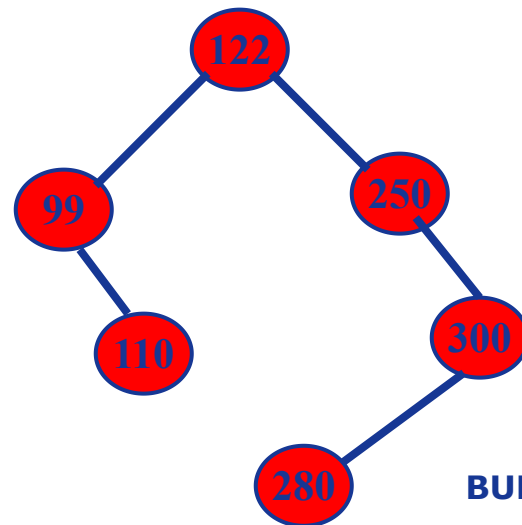
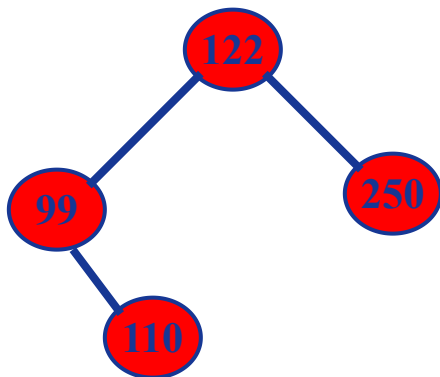
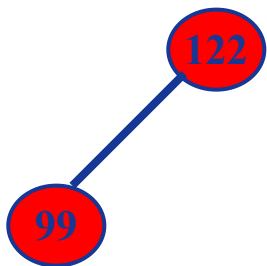
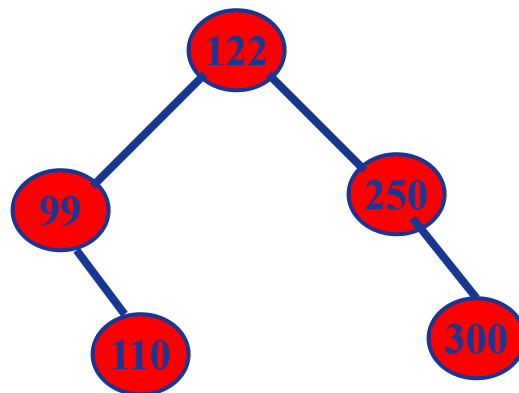
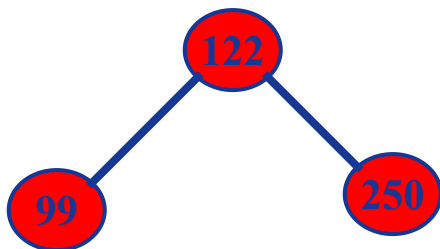
```
{
    if (T==NULL)
    {
        T = (SearchTree)malloc( sizeof(struct TreeNode));
        if (T == NULL)  FatalError("out of space!")
        else
        {
            T->data = X;
            T->left = T->right =NULL;
        }
    }
    else if (X< T->data)
L8  T->left = Insert(X, T->left);
    else if (X> T->data)
L10 T->right = Insert(X, T->right);
    //else X已经在树T中，不做任何操作；
    return T;
}
```

[生成]

[算法步骤] 反复执行以下操作

- a. 读入一个记录，设其关键字为K；
- b. 调用查找算法，确定插入位置；
- c. 调用插入算法，实施插入结点的操作；

例：将序列122、99、250、110、300、280 作为二叉排序树的结点的关键字值，生成二叉排序树。

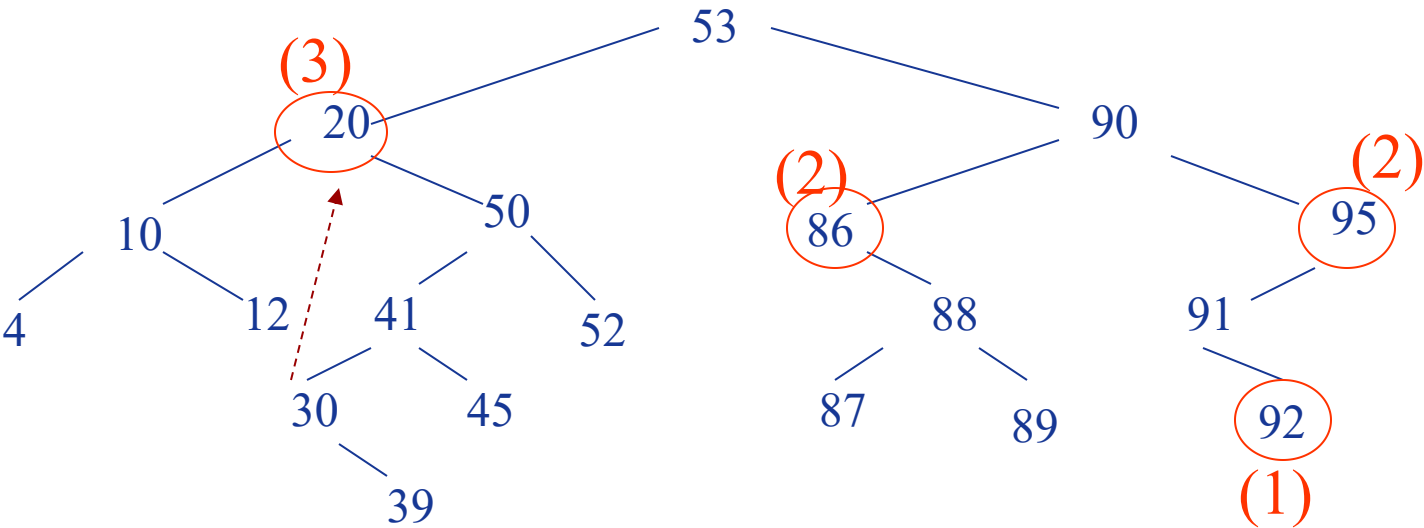


[删除]

依据被删除值为X的结点p的不同情况分析：

- 1. p是叶子结点：修改其双亲指针即可
- 2. p只有左孩子：用p的左子树代替以p为根的子树
- 3. p只有右孩子：用p的右子树代替以p为根的子树
- 3. p有两个孩子：找到p的序后继(或前趋)结点tmp，
tmp的数据复制给p，
删除只有右子(或左子)/无孩子的tmp

例：



```
SearchTree Delete(elemtype X, SearchTree T)
{
    Position tmp;
    if (T == NULL) Error("Element Not Found!")
    else if (X < T->data)    T->left = Delete(X, T->left); //go left
    else if (X > T->data)    T->right = Delete(X, T->right); //go right
    else if (T->left && T->right) //two children;
    {
        tmp = FindMin(T->right); //找右子树最小值
        T->data = tmp->data;
        T->right = Delete(T->data, T->right);
    }
    else //one children;
    {
        tmp = T;
        if (T->left == NULL) T = T->right;
        else if (T->right == NULL) T = T->left;
        free(tmp);
    }
    return T;
}
```

[4. 性能分析]

- ❖ 给定树的形态，等概率查找成功时的 $ASL = \sum c_i / n$
 - 最差(单支树): $(n+1)/2$
 - 最好(类似折半查找的判定树): $\log_2(n+1)-1$
 - 随机: $\leq 1+4\log_2 n$
- ❖ 关键字有序出现时，构造出“退化”的二叉排序树，树深为 n ，各种操作代价 $O(n)$ 。避免方法：采用平衡二叉树

8.7 平衡二叉树(AVL树)

BUPT

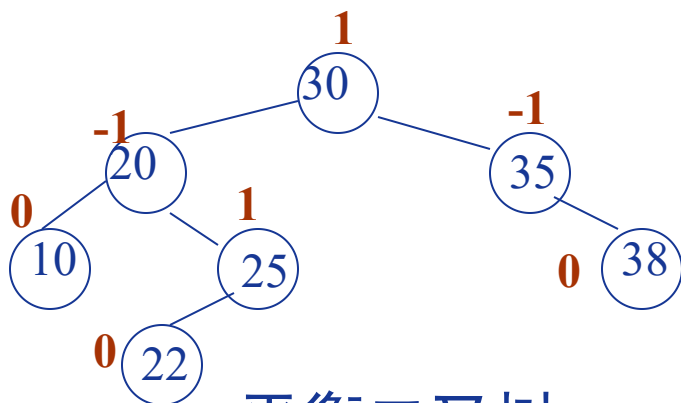
[1. 定义]

平衡二叉树或者是空树，或者是满足如下性质的**二叉排序树**：

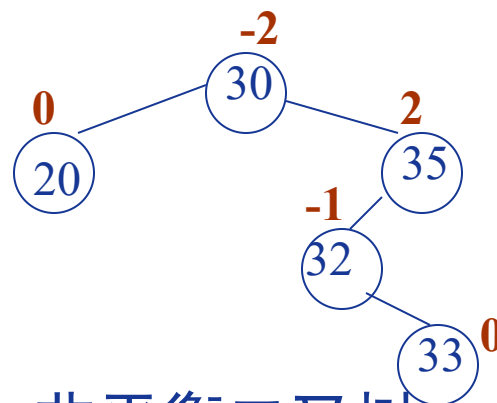
- 1) 它的左、右子树的高度之差的绝对值不超过1；
- 2) 其左右子树本身又各是一棵平衡二叉树。

二叉树上结点的**平衡因子**：

该结点的左子树高度减去右子树的高度。



平衡二叉树



非平衡二叉树

平衡二叉树：每个结点的平衡因子都为 +1、-1、0 的二叉排序树。

[2.结点的存储结构]

left	bf	data	right
------	----	------	-------

left: 左孩子指key

right: 右孩子指针

bf: 平衡因子

data: 数据元素

Struct TreeNode

```
{
    Struct TreeNode * left;
    Struct TreeNode * right;
    int bf;
    elemtype data;
};
typedef struct TreeNode* AVLTreeB;
typedef struct TreeNode* PositionB;
```

依照定义的格式，常用形式

left	height	data	right
------	--------	------	-------

left: 左孩子指key

right: 右孩子指针

height: 以该结点为根的子树高度

data: 数据元素

Struct TreeNode

```
{
    Struct TreeNode * left;
    Struct TreeNode * right;
    int height;
    elemtype data;
};
typedef struct TreeNode* AVLTreeH;
typedef struct TreeNode* PositionH;
```

非常用形式，但实现操作更简便

利用平衡因子求高度

```
int Height(PositionB p)
{
    int h=0;
    while (p!=NULL)
    {
        h++;
        if(p->bf >= 0) p = p->left;
        else p = p->right;
    }
    return h;
}
```

高度

```
int Height(PositionH p)
{
    if(p == NULL)
        return 0;
    else
        return p->height;
}
```

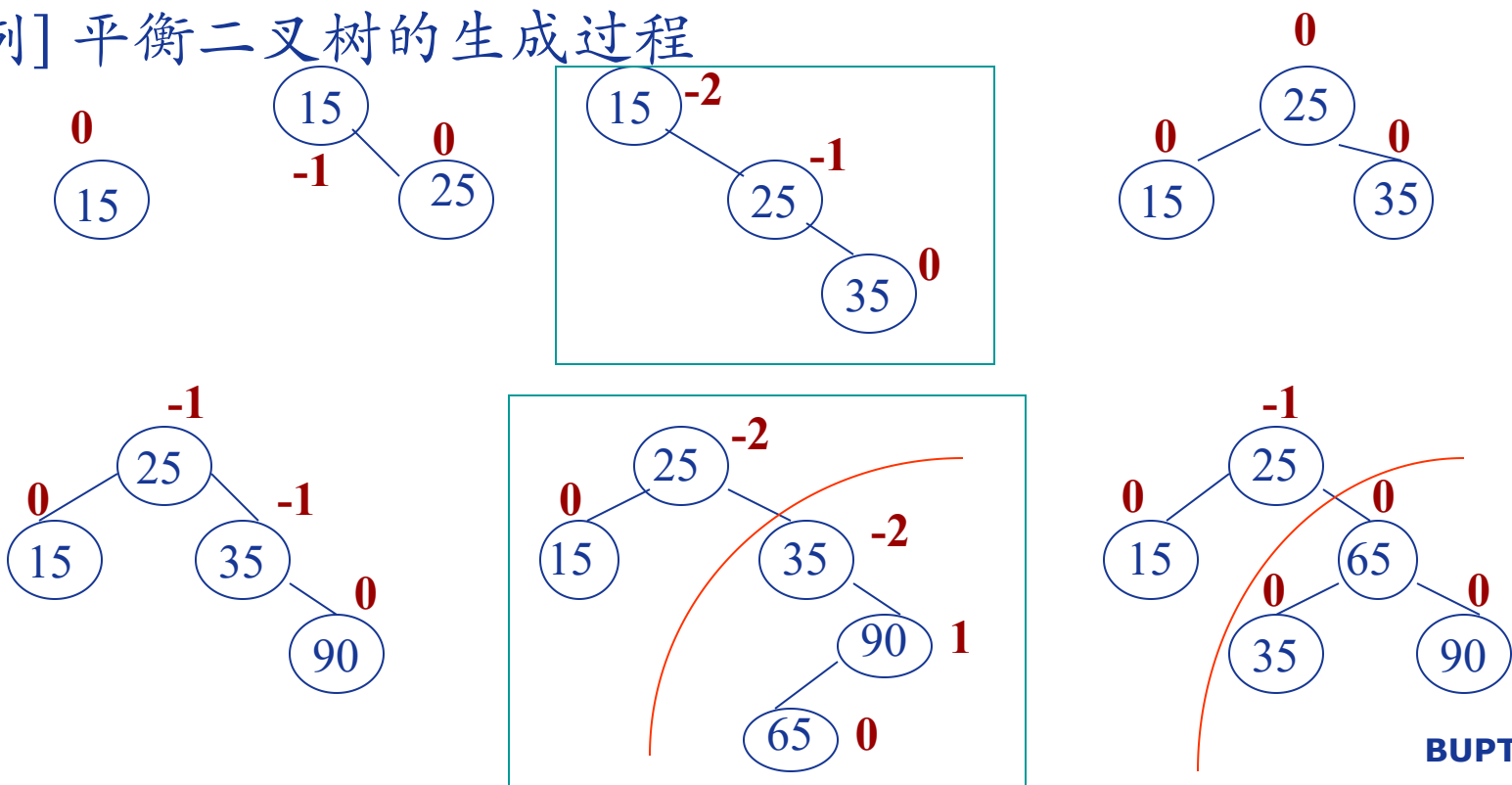
[4. 在平衡二叉树上的操作]

[查找]

查找方法同二叉排序树。

[插入] 插入新结点之后仍应保持平衡二叉树的性质不变。

[例] 平衡二叉树的生成过程



[调整范围的确定]

插入结点后，找到离插入结点最近且平衡因子绝对值超过1的祖先结点（**危机节点**），则以该**危机节点**为根的子树将是可能不平衡的**最小子树**，可将重新平衡的范围局限于这棵子树。

[调整的类型]

LL型-表示新插入结点在危机结点的**左子**的**左子树**上

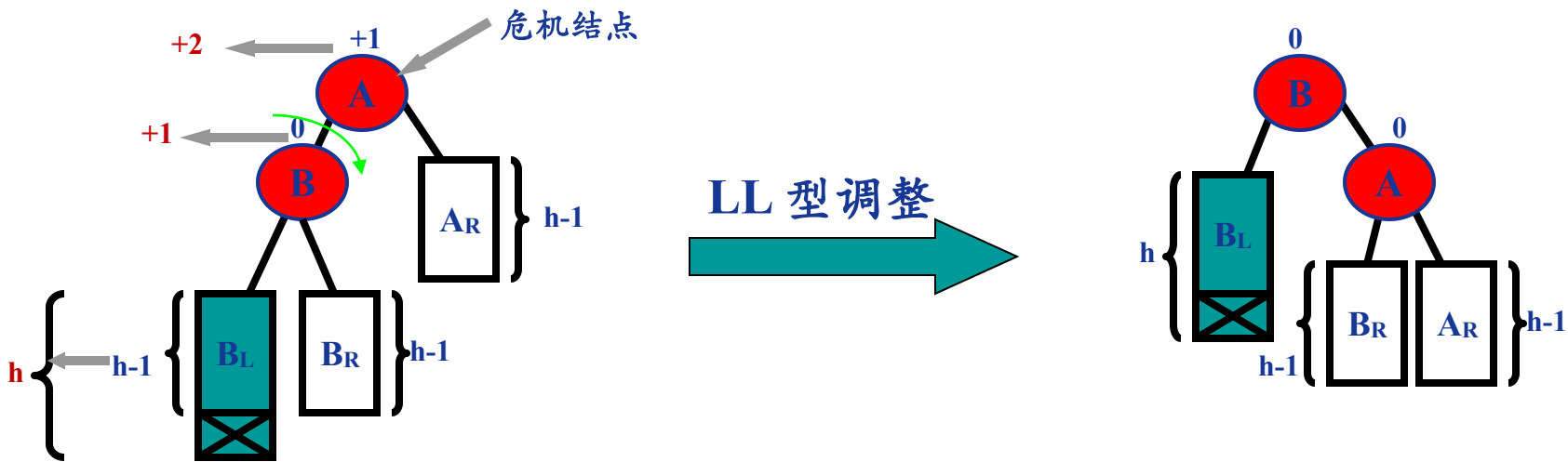
LR型-表示新插入结点在危机结点的**左子**的**右子树**上

RL型-表示新插入结点在危机结点的**右子**的**左子树**上

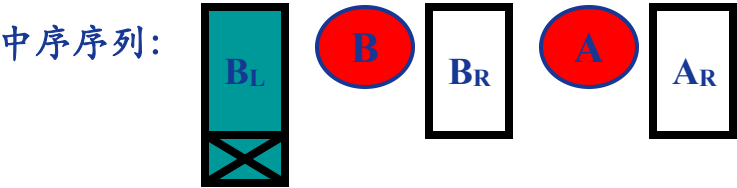
RR型-表示新插入结点在危机结点的**右子**的**右子树**上

[调整的方法]

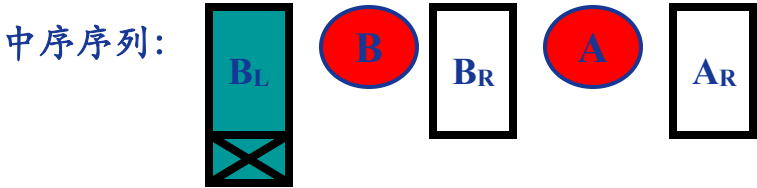
LL型平衡旋转——一次顺时针旋转



调整前：高度为 $h + 1$



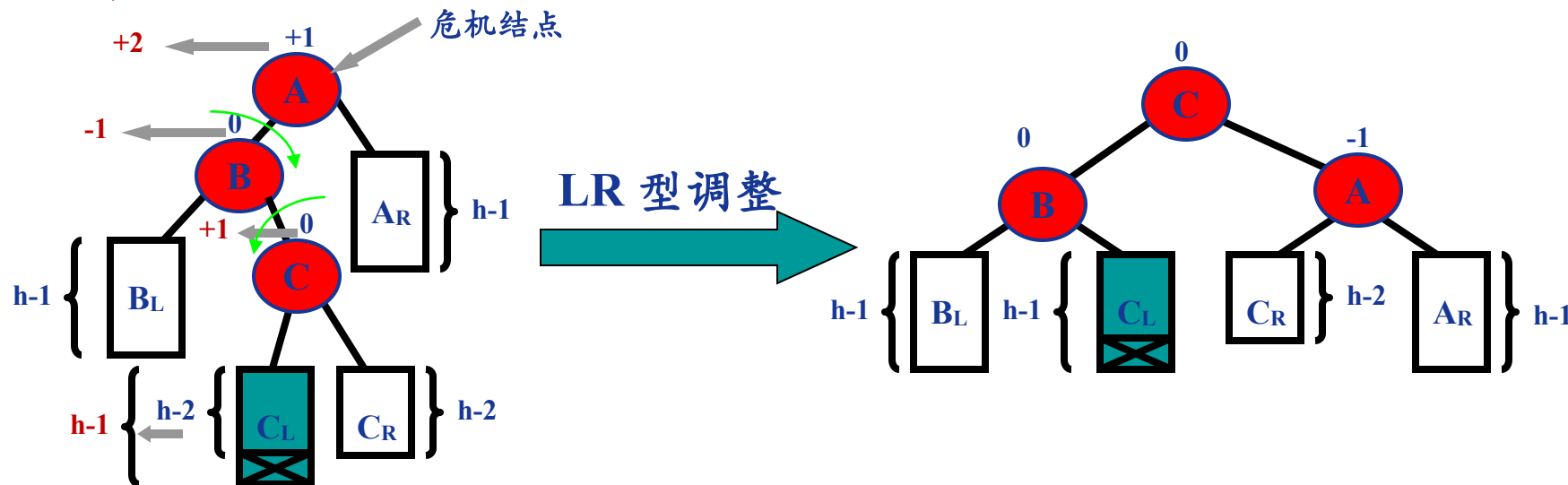
调整后：高度为 $h + 1$



注意：调整后   平衡因子为 0

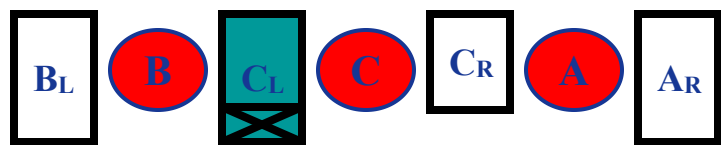
LR型平衡旋转——一次逆时针旋转+一次顺时针旋转

情形A



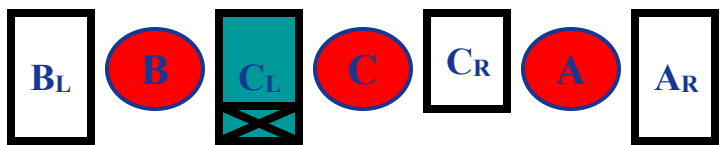
调整前：高度为 $h + 1$

中序序列：



调整后：高度为 $h + 1$

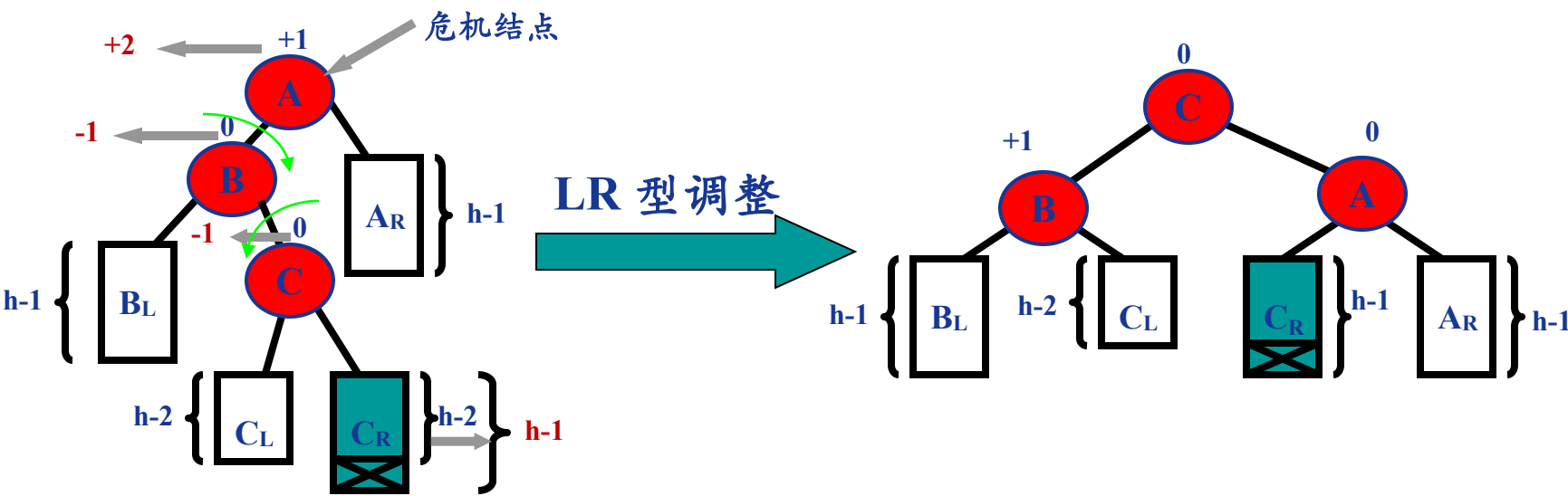
中序序列：



注意：调整后 **B C A** 平衡因子为 $+1, 0, 0$

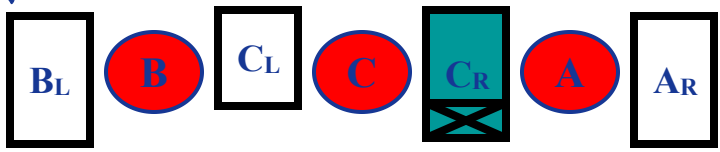
LR型平衡旋转——一次逆时针旋转+一次顺时针旋转

情形B



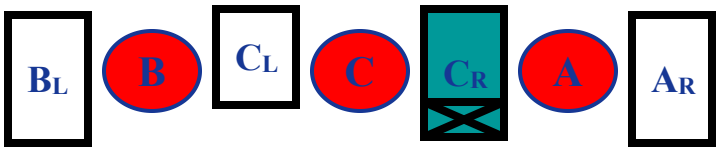
调整前：高度为 $h + 1$

中序序列：



调整后：高度为 $h + 1$

中序序列：



注意：改组后 B C A 平衡度为 +1,0,0

LR型平衡旋转——一次逆时针旋转+一次顺时针旋转

情形C

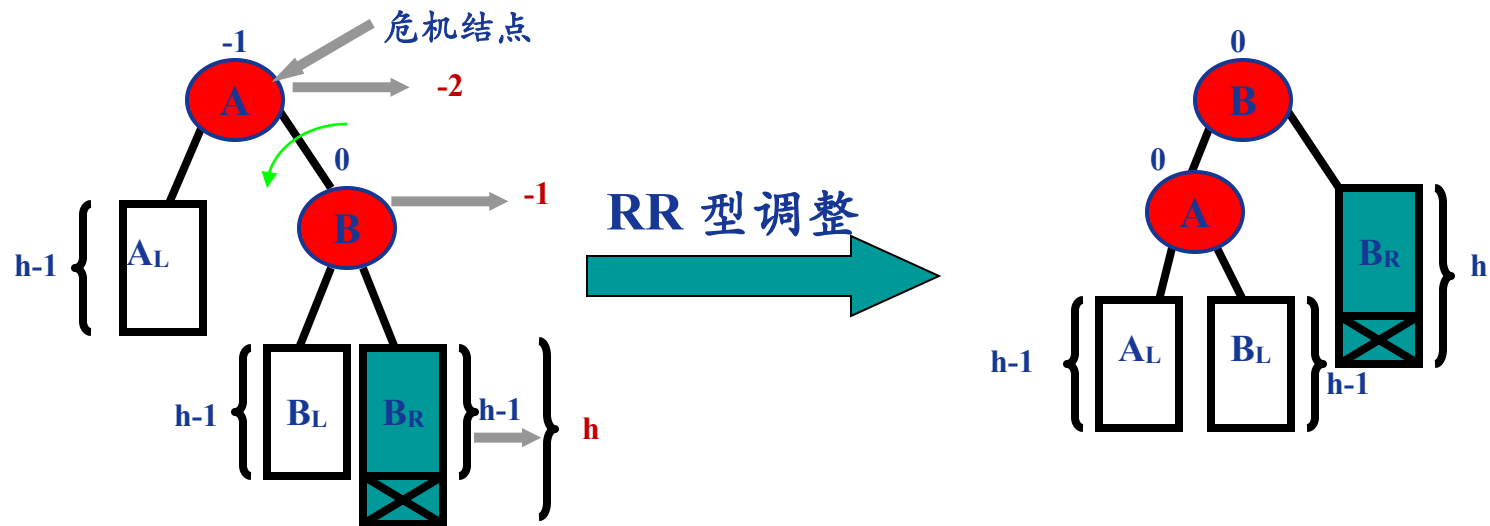


注意：调整后
平衡因子为 0,0,0

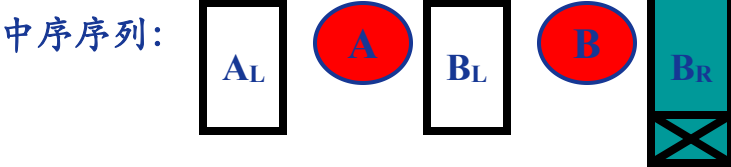
调整前：高度为 2
中序序列：B C A

调整后：高度为 2
中序序列：B C A

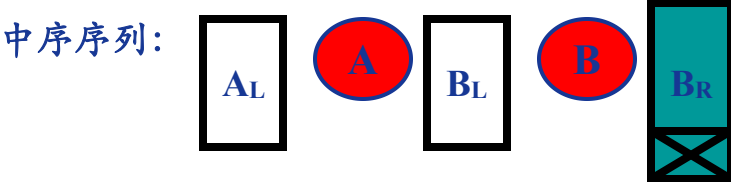
RR型平衡旋转——一次逆时针旋转



调整前：高度为 $h + 1$

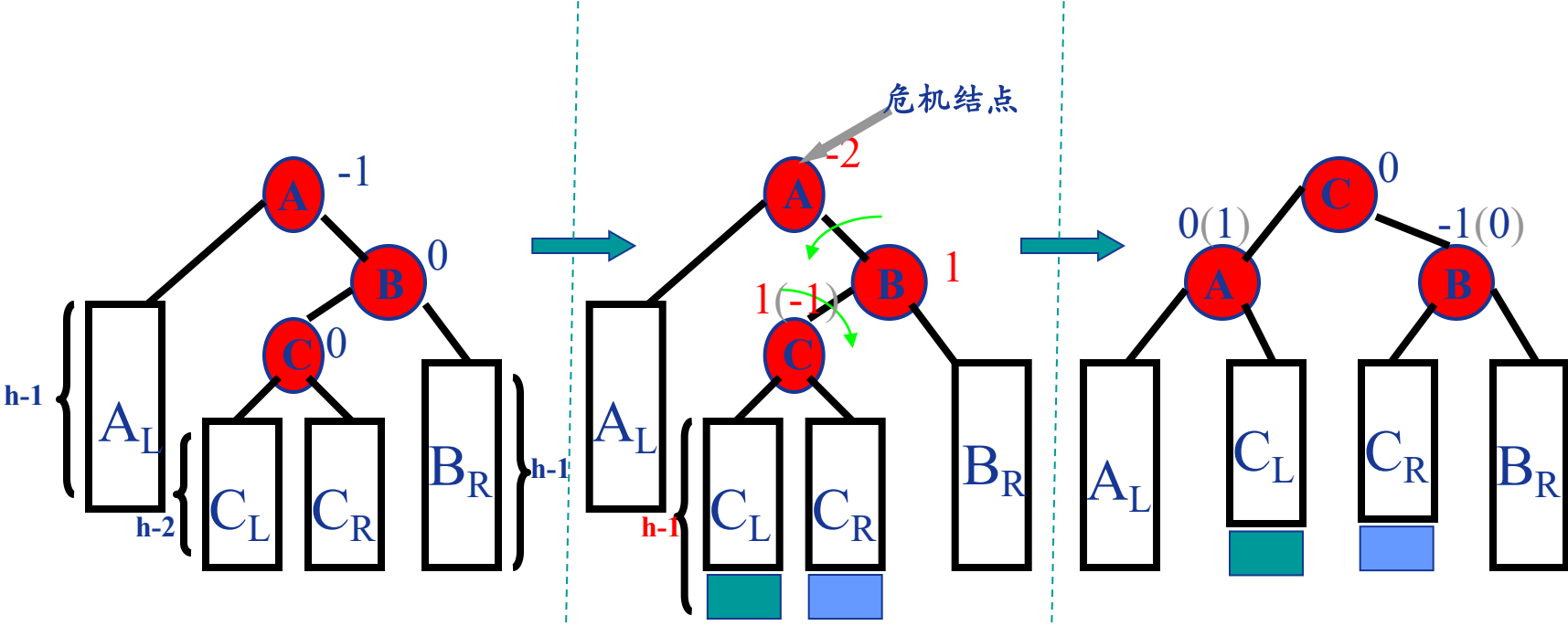


调整后：高度为 $h + 1$



注意：调整后   平衡因子为 0

RL型平衡旋转——一次顺时针旋转+一次逆时针旋转



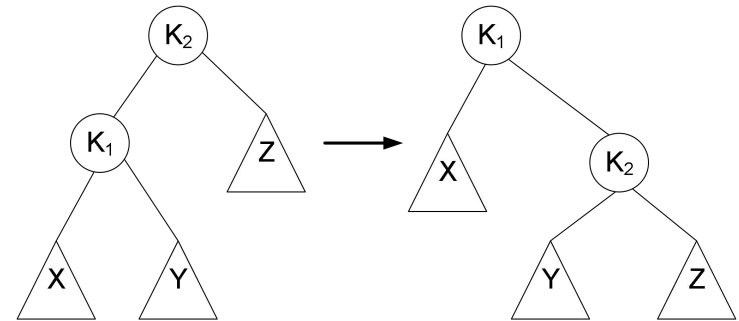
[AVL树插入结点]

AvlTreeH Insert(elemtype X, AVLTreeH T) //插入值为X的结点, T是危机结点为根的子树

```
{
    if (T==NULL) {
        T = (AvlTreeH) malloc(sizeof(AvlNode));
        if (T==NULL) FatalError("Out of space!")
        else {
            T->data =X; T->height = 1;
            T->left = T->right = NULL;
        }
    }
    else if (X < T->data) {
        T->left = Insert(X, T->left);
        if(Height(T->left) - Height(T->right) == 2)
            if(X < T->left->data) T= SingleRotateWithLeft(T); //LL型, 单次顺时针旋转
            else T = DoubleRotateWithLeft(T); // LR型, 一次顺时针+一次逆时针旋转
    }
    else if (X>T->data ) {
        T->right = Insert(X, T->right);
        if(Height(T->right) - Height(T->left) == 2)
            if(X > T->right->data) T= SingleRotateWithRight(T); //RR型, 单次逆时针旋转
            else T = DoubleRotateWithRight(T); // LR型, 一次逆时针+一次顺时针旋转
    }
    T->height = MAX(Height(T->left) , Height(T->right) ) + 1;
    return T;
}
```

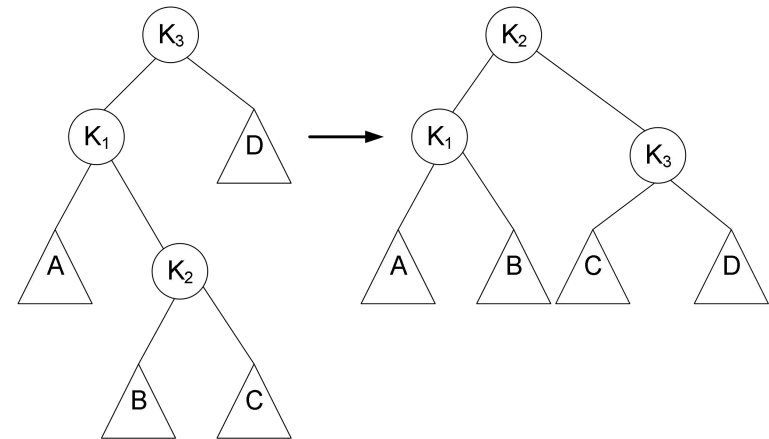
```
PositionH SingleRotateWithLeft(PositionH k2)
```

```
{
    PositionH k1;
    k1 = k2->left;
    k2->left = k1->right;
    k1->right = k2;
    k2->height = Max(Height(k2->left), Height(k2->right ))+1;
    k1->height = Max(Height(k1->left), k2->height )+1;
    return k1;
}
```



```
PositionR doubleRotateWithLeft(PositionH k3)
```

```
{
    //k1 和 k2之间逆时针旋转
    k3->left = SingleRotateWithRight(k3->left);
    //k3 和 k2之间顺时针旋转
    return SingleRotateWithLeft(K3);
}
```



SingleRotateWithRight和doubleRotateWithRight为对偶情况

[删除] (思路同插入)

- ❖ 将删除结点q转化为q最多有一个孩子的情形，即若q有两个孩子，则以其中序前驱/后继结点r取代它，删除r；
- ❖ 若树的平衡性被破坏，利用单一/双重旋转恢复。

[性能]

定理：一个具有n个结点的平衡二叉树形，其高度h为

$$\log_2(n+1) \leq h \leq 1.4404\log_2(n+2)-0.328$$

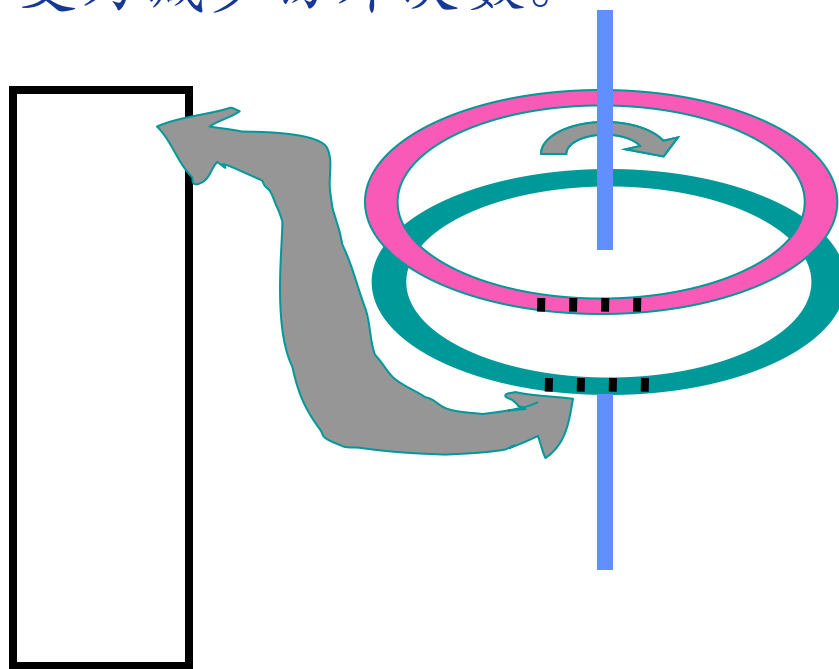
结论：最坏情况下，AVL树的高度约为 $1.44\log_2 n$ ，而完全平衡的二叉树高度约为 $\log_2 n$ ，因此AVL树是接近最优的，其平均查找长度与 $\log_2 n$ 同数量级。

8.7 B树与B+树

BUPT

[采用B与B+树的意义]

大量数据存放在外存中，由于是海量数据，不可能一次调入内存。因此，要多次访问外存，速度慢。所以，主要矛盾变为减少访外次数。

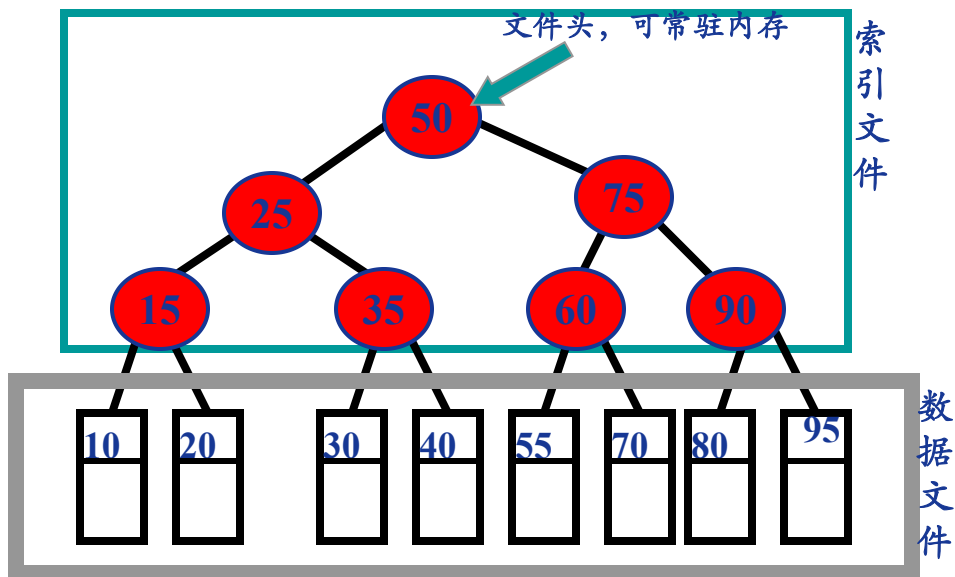


内存

BUPT SCST

用二叉树组织文件，需访问外存次数太多。

如：当文件的记录个数为 100,000 时，要找到给定关键字的记录，需访问外存 17 次 ($\log_{100,000}$)，太多了！



文件访问示意图：索引文件存在内存、数据文件存在盘上

[B-树]-AVL树推广到多叉树

B-树是一种平衡的多路查找树。应用于文件系统。

1. 定义

一棵 **m 阶B树**，或为空树，或为满足下列特性的 m 叉树：

- ① 树中每个结点最多有 **m** 棵子树；
- ② 根结点或者是叶子结点，或者其子树数在 **2~m** 之间；
- ③ 除根之外的所有非叶子结点子树数在 **$\lceil m/2 \rceil \sim m$** 之间；
- ④ 所有非终端结点包含 **$(n, A_0, K_1, A_1, K_2, \dots, K_n, A_n)$** 信息数据；其中 n 为结点中关键字个数 (**$n \leq m-1$**)， A_i 为指向子树的指针， K_i 为关键字。
- ⑤ 所有叶子结点都在同一层上，且不带信息。

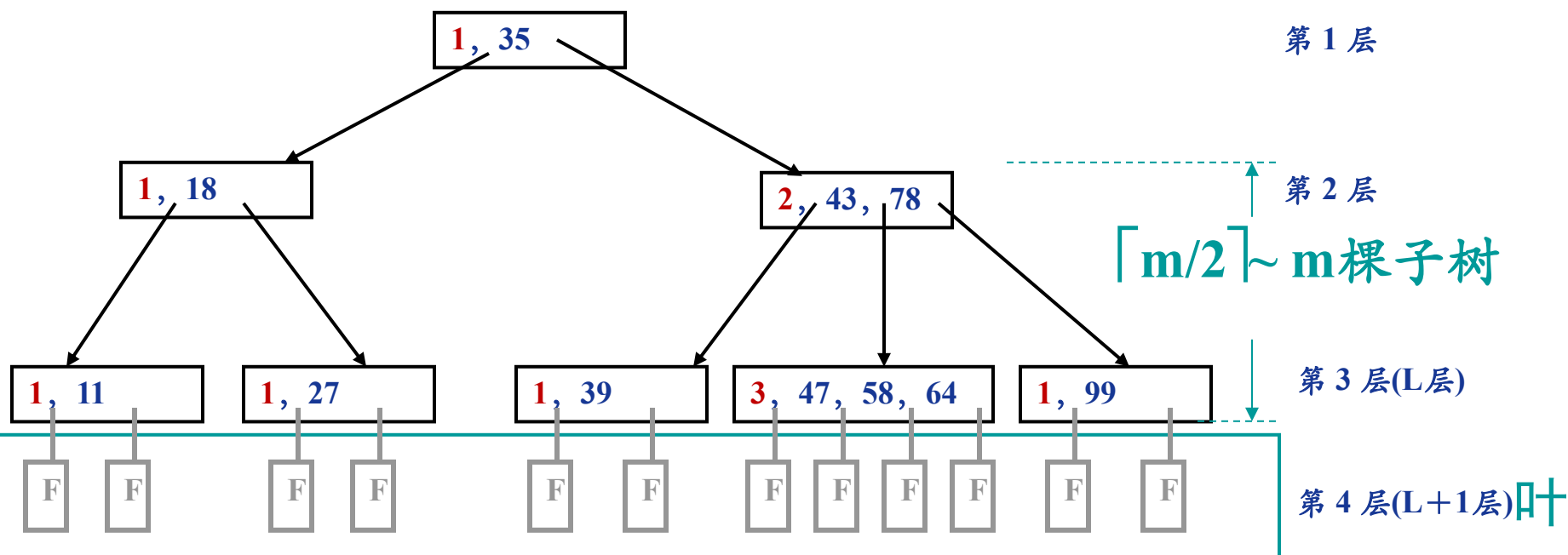
4阶B树常被称为2-3-4树，3阶B树常被称为2-3树

2. B-树结点结构

$(n, A_0, K_1, A_1, K_2, A_2, \dots, K_n, A_n)$

- n : 关键字的个数
- A_0 : $<K_1$ 的结点组成的子树地址 (指在该 B 树中)
- K_1 : 关键字
- A_2 : $>K_1$ 且 $<K_2$ 的结点组成的子树地址 (指在该 B 树中)
- 其余类推
- A_n : $>K_n$ 的结点组成的子树地址 (指在该 B 树中)
- $K_1 < K_2 < \dots < K_n$

例如：**4阶B树**，又称**2-3-4树**，要么是空树，要么包含三类结点：**2-结点**：1个关键字，一个指向带有较小关键字的左指针，一个指向带有较小关键字子树的左指针，一个指向带有较大关键字子树的右指针；**3-结点**：2个关键字，一个指向带有较小关键字的左指针，一个指向带有该结点两个关键字定义区间之间的子树的指针，一个指向带有较大关键字子树的右指针；**4-结点**：三个关键字、四个指针，分别指向三个关键字所含的四个值域区间的子树。



设关键字的总数为 N ，求 m 阶 B 树的最大层次 L 。

层次	结点数	关键字的个数（最少）
1	1	1
2	2	$2(\lceil m/2 \rceil - 1)$
3	$2(\lceil m/2 \rceil)$	$2(\lceil m/2 \rceil)(\lceil m/2 \rceil - 1)$
4	$2(\lceil m/2 \rceil)^2$	$2(\lceil m/2 \rceil)^2(\lceil m/2 \rceil - 1)$
...	...	...
L	$2(\lceil m/2 \rceil)^{L-2}$	$2(\lceil m/2 \rceil)^{L-2}(\lceil m/2 \rceil - 1)$
$L+1$	$2(\lceil m/2 \rceil)^{L-1}$ 叶子	

所以， $N = 2(\lceil m/2 \rceil)^{L-1} - 1$

故： $L = \log_{\lceil m/2 \rceil}((N+1)/2) + 1$

设 $N = 1000,000$ 且 $m = 256$ ，

则 $L \leq 3$ ；最多 3 次访问外存可找到所有的记录。

3.B树查找过程

类似于二叉排序树的查找。如查找关键字为 KEY 的记录。

- 从根开始查找，如果 $K_i = KEY$ 则查找成功。
- 若 $K_i < KEY < K_{i+1}$ ；查找 A_i 指向的结点
- 若 $KEY < K_1$ ；查找 A_0 指向的结点
- 若 $KEY > K_n$ ；查找 A_n 指向的结点
- 若 找到叶子，则查找失败。

4.m阶B-树插入过程

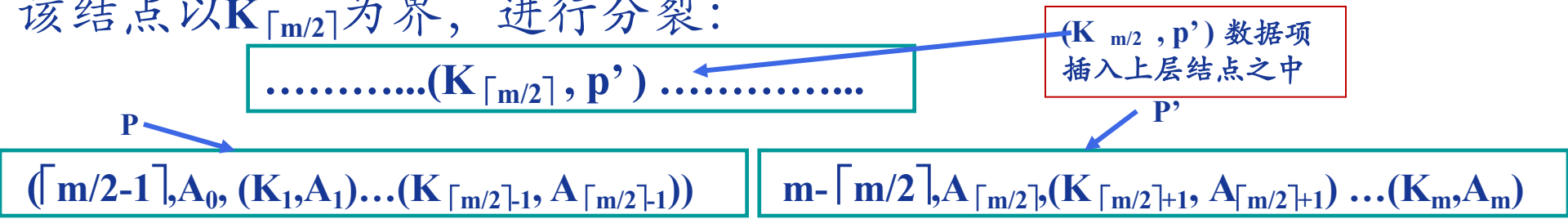
- 1、利用查找过程确定第 L 层上数据元素的插入位置（注意：第 L+1 层为叶子结点）
- 2、找到插入位置后，将关键字和其它信息按序插入到相应结点上。
- 3、若插入新数据元素后，被插入结点的关键字个数为m, 则该结点满。必须分裂成两个结点。否则不满结束。

如结点原为：

$$(m-1, A_0, (K_1, A_1), (K_2, A_2), \dots (K_{m-1}, A_{m-1}))$$

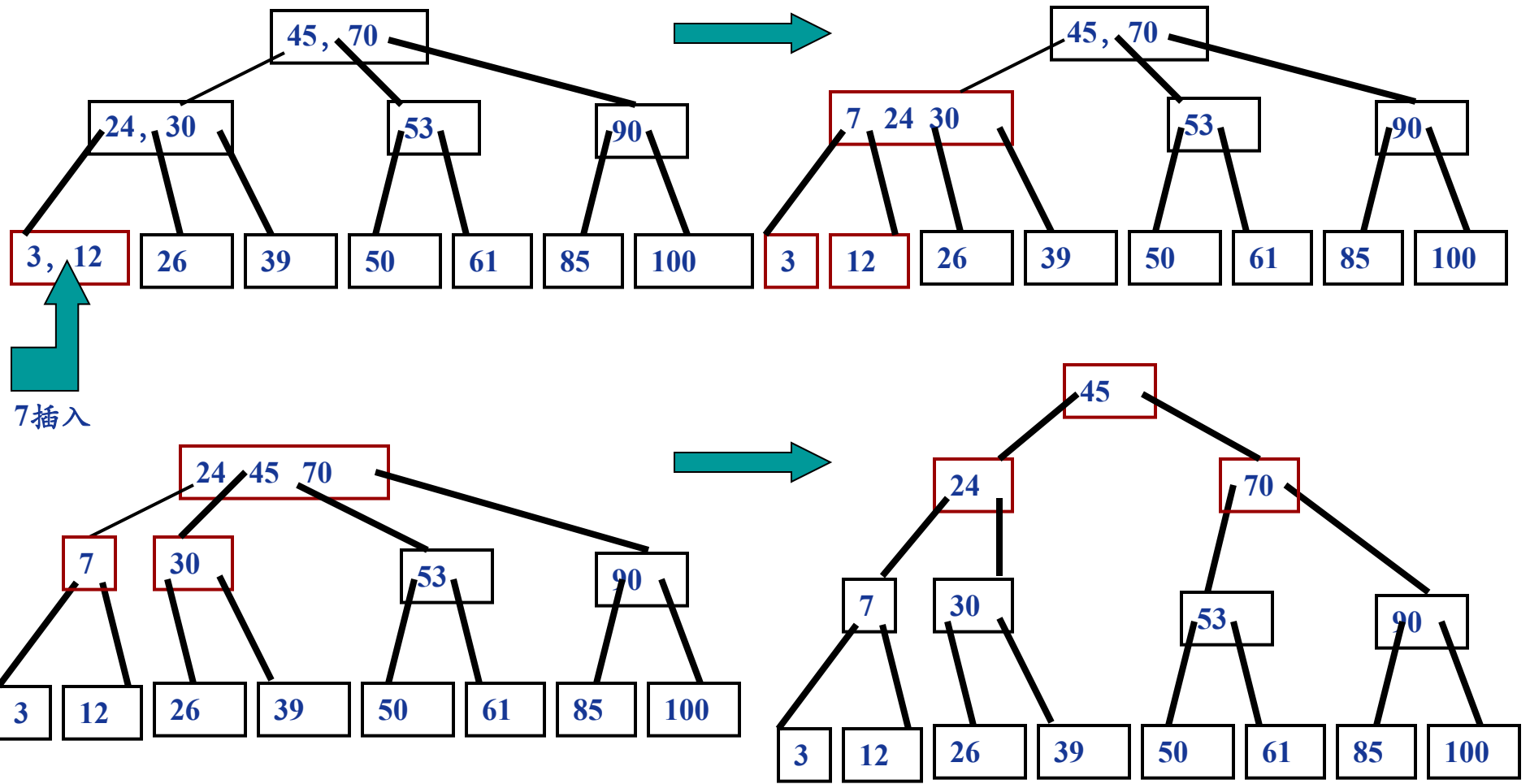
插入一个关键字之后变为： $(m, A_0, (K_1, A_1), (K_2, A_2), \dots (K_m, A_m))$

该结点以 $K_{\lceil m/2 \rceil}$ 为界，进行分裂：



生成新结点 p' ，将原结点的后半部分复制过去。若分裂一直进行到根结点，树可能长高一层。

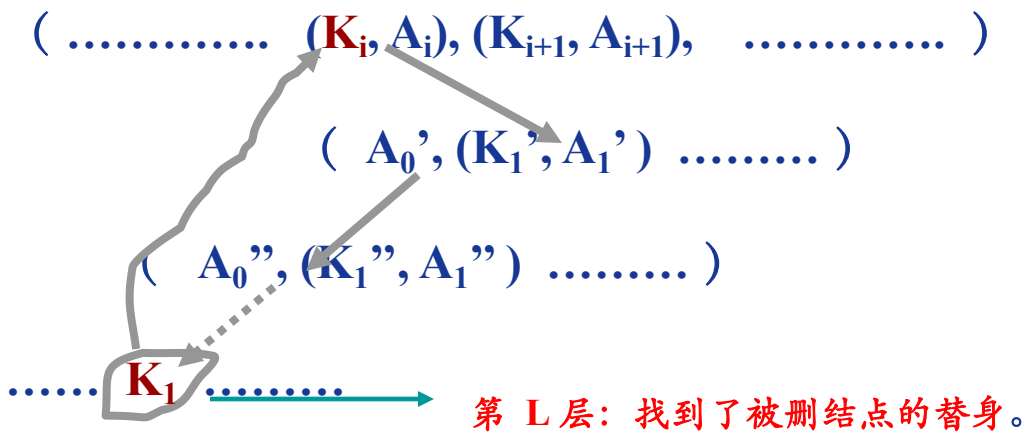
例：2-3树的插入key=7



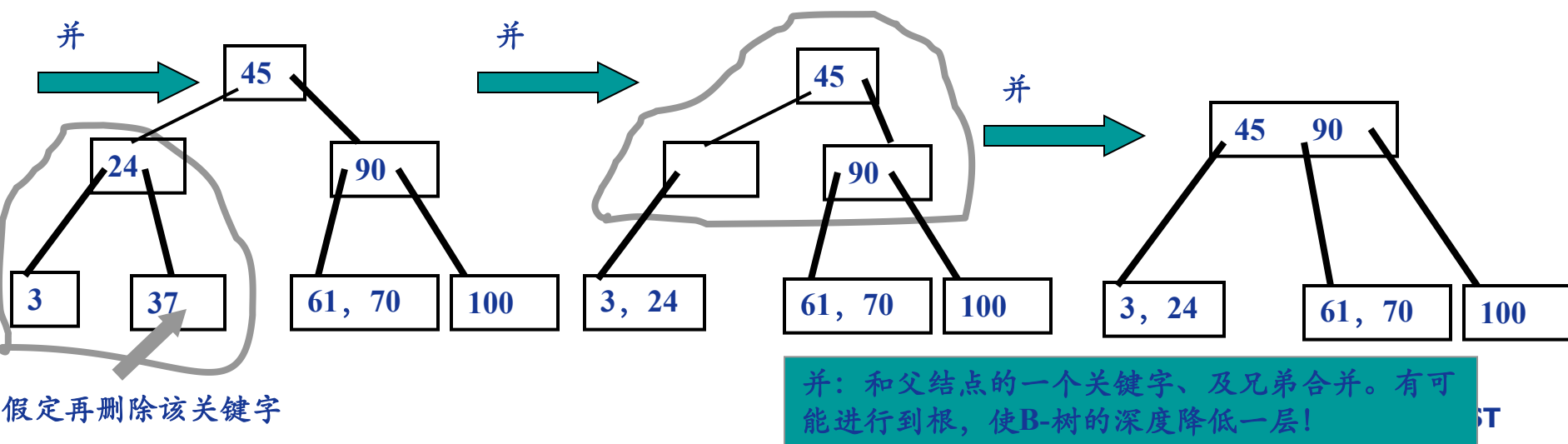
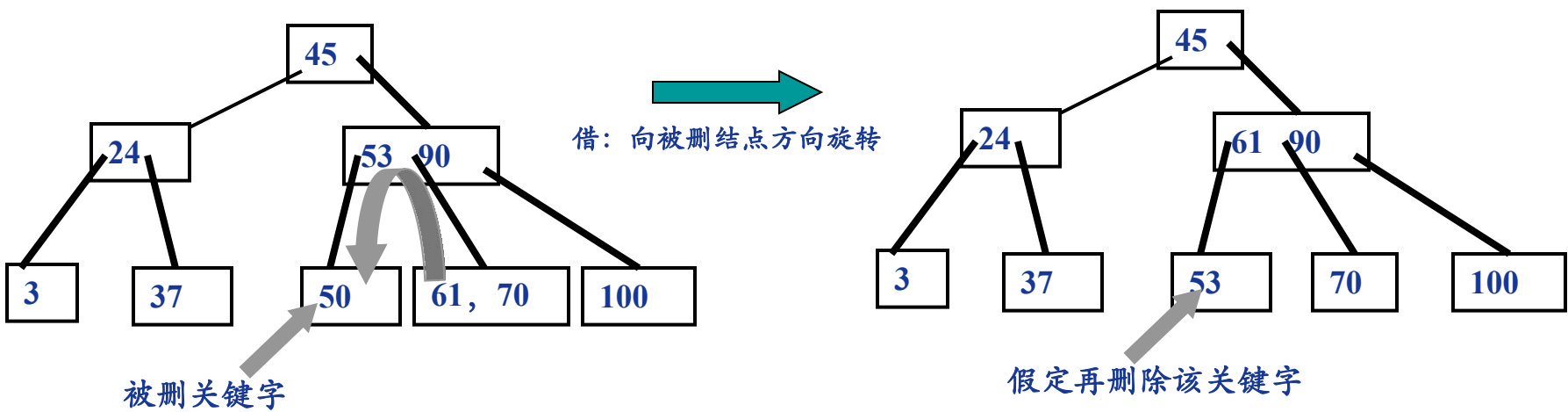
5.B-树删除过程

- 1、查找具有给定键值的关键字 K_i
- 2、如果在第 L 层，可直接删除（注意：第 $L+1$ 层为叶子结点），转 4。
- 3、否则，则首先生成“替身”。用该关键字的右边子树中的最左面的结点的关键字取代。然后，删除第 L 层上的该关键字。
- 4、从第 L 层开始进行删除。
 - A、不动：若删除关键字值的那个结点的关键字的个数仍处于 $\lceil m/2 \rceil - 1$ 和 $m-1$ 之间。则结束。
 - B、借：若删除关键字值的那个结点的关键字的个数原为 $\lceil m/2 \rceil - 1$ 。而它们左\右兄弟结点的关键字个数 $> \lceil m/2 \rceil - 1$ ；则借一关键字过来，结束。
 - C、并：若该结点的左\右兄弟结点的关键字的个数为 $\lceil m/2 \rceil - 1$ ；则执行合并结点的操作。

如结点原为：



例如：2-3树的删除操作。



[B+树]

B+树是一种B-树的变形树。应用于索引顺序文件系统。

1. m阶B⁺树与B-树的不同之处

1)有 n 棵子树的结点中有 n 个关键字;

2)非叶结点可以看成是索引部分 **索引集**

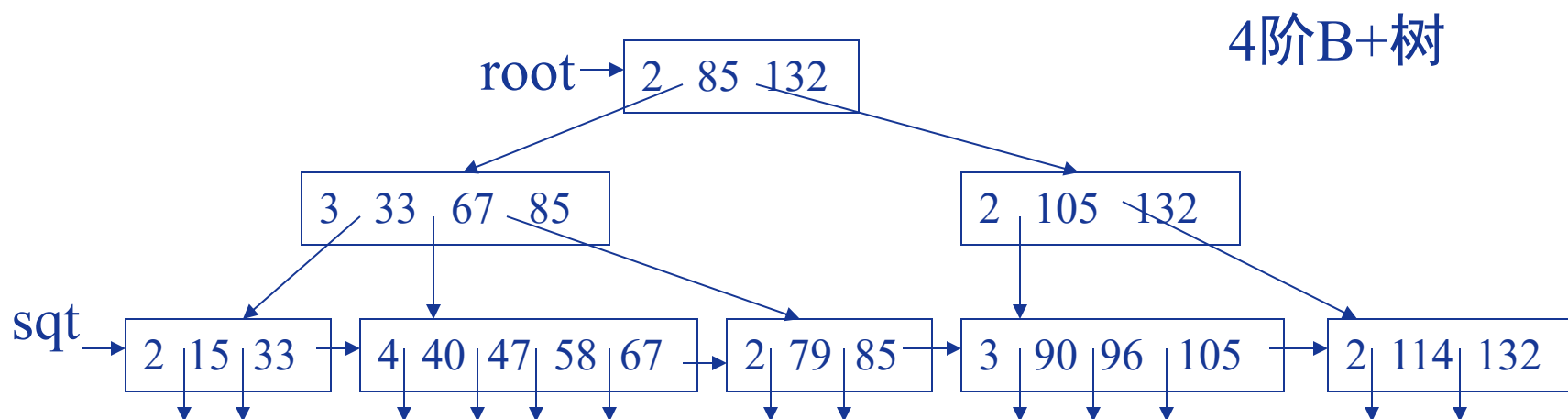
A_i : 第 i 个子结点的指针

K_i : 第 i 个子结点的最大 (或最小) 关键字

3)所有叶子结点中包含了全部关键字的信息及指向这些关键字记录的指针, 且叶子结点以关键字大小自小至大顺序链接; **数据集**

[结点结构]

- 非叶结点 ($A_1, K_1, \dots, A_i, K_i, \dots, A_n, K_n$) 索引集
 A_i : 第*i*个子结点的指针
 K_i : 第*i*个子结点的最大 (或最小) 关键字
- 叶结点
 全部关键字及指向关键字记录的指针 数据集



2. B+树上的基本运算

1)查找

- ❖ 方式1: 从根结点开始, 利用索引集结构, 向下查找直到叶子结点
- ❖ 方式2: 从最小关键字开始, 沿叶结点数据集的链结构顺序查找

2)插入

仅在叶子结点上进行, 关键字个数大于 m 则分裂

3)删除

也仅在叶子结点上进行, 关键字个数小于 $\lceil m/2 \rceil$ 时, 需进行合并

8.8 哈希表

BUPT

[哈希表的基本思想]

在记录的存储地址和它的关键字之间建立一个确定的对应关系；这样，理想状态不经过比较，一次存取就能得到所查元素。

[术语]

哈希表 一个有限的连续的地址空间，用以容纳按哈希地址存储的记录。

哈希函数 记录的存储位置与它的关键字之间存在的一种对应关系。 $\text{Loc}(r_i) = \mathbf{H}(\text{key}_i)$

冲突 对于 $\text{key}_i \neq \text{key}_j$, $i \neq j$, 出现的 $H(\text{key}_i) = H(\text{key}_j)$ 的现象。

同义词 在同一地址出现冲突的各关键字。

哈希(散列)地址 根据设定的哈希函数 $H(\text{key})$ 和处理冲突的方法确定的记录的存储位置。

装填因子 表中填入的记录数 n 和哈希表表长 m 之比。

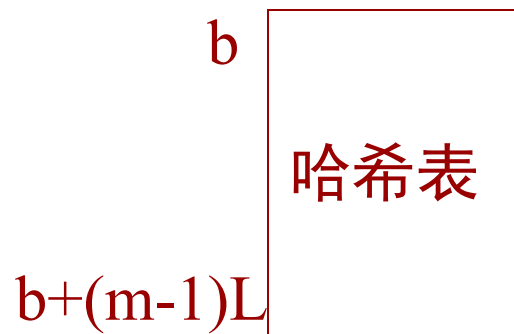
$$\alpha = n/m$$

[设计哈希表的过程]

1) 明确哈希表的地址空间范围。即确定哈希函数的值域。

2) 选择合理的哈希函数。该函数要保证所有可能的记录的哈希地址均在指定的值域内，并使冲突的可能性尽量小。

3) 设定处理冲突的方法。



[哈希函数的基本构造方法]

构造原则： $\left\{ \begin{array}{l} \text{算法简单，运算量小；} \\ \text{均匀分布，减少冲突。} \end{array} \right.$

[1. 直接定址法]

$H(\text{key}) = a * \text{key} + b$ $a、b$ 为常数

e.g: 令： $a、b$ 为 1，有两个关键字 k_1, k_2 值为 10、1000；选 10、1000 作为存放地址。

特点：计算简单，冲突最少。

[2. 数字分析法/基数转换法]

取关键字在某种进制下的若干数位作为哈希地址。对于非数字关键字要先转成非负整数。

e.g: $\text{key} = 236076$ 基数为 10，看成是 13 进制的。

则： $(236075)_{13} = 8 \ 4154 \ 7$ ；选取 4154 作为散列地址(假设表长 $m = 10000$)。

两种非数字关键字转换成非负整数的方法

方法一：把每个字符先转成唯一整数，再这些整数相加之和作为hash地址，每个字符对应整数0~255，函数返回值不超过8bit。

```
unsigned int StringToInt(char *key)
{
    int number = 0;
    while(*key)
        number += *key++;
    return number;
}
```

方法二：每隔一个字符左移8位后，与已得结果相加。

```
unsigned int StringToInt(char *key)
{
    int number = 0;
    while(*key)
        number += *key++;
    if(*key)
        number += ((int)*key++) << 8;
    return number;
}
```

[3. 平方取中法]

取关键字平方后的中间几位作为哈希地址。

e.g: $(4731)^2 = 223\ 82\ 361$; 选取 82 (假设 $m = 100$) 。

[4. 随机数法]

$H(\text{key}) = \text{random}(\text{key})$

特点：较适于关键字长度不等时的情况。

[5. 折叠法]

将关键字分割成位数相同的几部分(最后一部分的位数可以不同)，然后取这几部分的叠加和(舍去进位)作为哈希地址。

e.g: key = 381, 412, 975

选取 768 或 570 作为散列地址 (假设 $m = 1000$) 。

$$\begin{array}{r}
 381 \\
 412 \\
 + 975 \\
 \hline
 1\ 768
 \end{array}$$

$$\begin{array}{r}
 975 \\
 214 \\
 + 381 \\
 \hline
 1\ 570
 \end{array}$$

[6. 除留余数法]

$$H(\text{key}) = \text{key} \bmod p \quad (p \leq m)$$

m: 哈希表的表长; p : 最好为素数

最常用, 余数总在 $0 \sim p-1$ 之间。通常 p 选 $< m$ 的最大质数

如: $m = 1024$, 则 $p = 1019$ 。

e.g: 选取 p 为质数的理由:

设 key 值都为奇数, 选 p 为偶数;

则 $H(\text{key}) = \text{key} \bmod p$, 结果为奇数, 一半单元被浪费掉。

设 key 值都为 5 的倍数, 选 p 为 95;

则 $H(\text{key}) = \text{key} \bmod p$, 结果为: 0、5、10、15、..... 90
奇数。4/5 的单元被浪费掉。

[*安全的哈希函数]

哈希函数常用于计算机安全领域，之一是消息验证。消息M通过不安全通道从A传到B，希望B确信接收到的信息就是原始消息，而不是伪造的消息。

解决方法：A使用安全通道传输消息M的哈希值 $h(M)$ 给B，仍然使用不安全通道传输消息M，B将收到的消息M'计算哈希值 $h(M')$ ，若 $h(M') \neq h(M)$ ，则为假消息。但M和M'若为同义词也能使得 $h(M') = h(M)$ ，因此选用的哈希函数应该对于已知M和h的恶意用户很难获得M的一个同义词(弱抗冲突)。

安全哈希函数还应该具有的特性：

- ①单向特性：给定一个c，难以通过计算找到k，满足 $h(k) = c$ 。
- ②强抗冲突：难以通过计算找到一对(x,y)，使得 $h(x) = h(y)$ 。
- ③H可被应用与任何大小的一组数据。
- ④H能产生固定长度的hash码。
- ⑤对任意给定的k， $h(k)$ 计算简单，容易实现。

[示例：Secure Hash Algorithm SHA-1]

- ①对输入信息进行预处理，如在末尾加一串0；使其长度表示为 $q \times 512$, q 为整数。
- ②初始化160为的输出缓冲器OB，它由5个32为寄存器ABCDE组成， $A=67452301$, $B=efcdab89$, $C=98badcfe$, $D=10325476$, $E=c3d2e1f0$
- ③for(int $i = 1$; $i \leq q$; $i++$) {
 B_i = 输入信息的第 i 个512数据块;
 $OB = F(OB, B_i)$;
 }
 ④输出OB;

⊕ 是异或操作

假设 $F=B \oplus C \oplus D$, $K_0=5a827999$

通过计算更新寄存器A-E的值，有

$B=67452301$, $D=98badcfe$, $E=10325476$ 。C

由B循环右移30位得到，

$B=11101111110011010101110001001$,

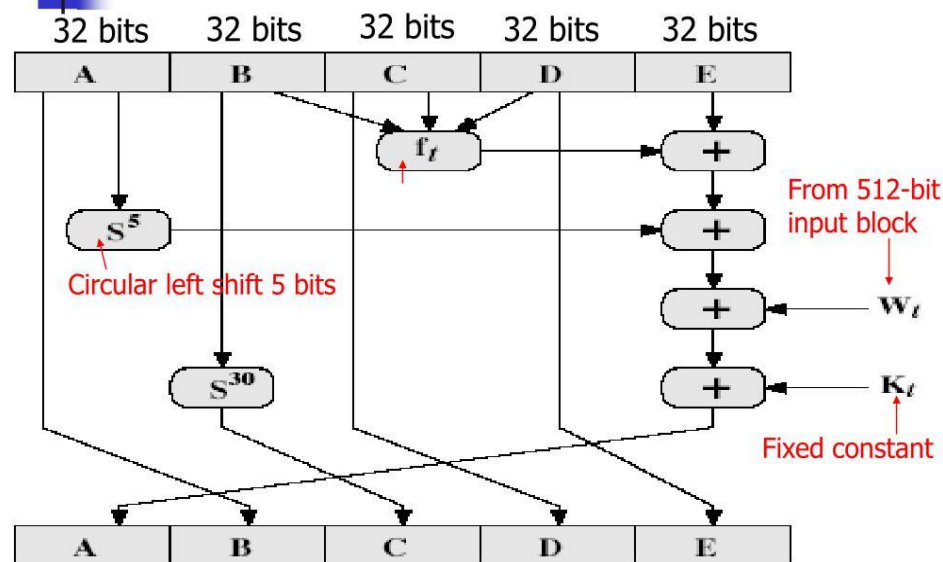
$C=01111011111100110110101011100010=7b3$

$6ae2$,将5个值相加得到A中新值，

$f(B,C,D)=B \oplus C \oplus D=67452301$; $s^5(A)=e8a460$

$2c$,相加取模 2^{32} 得 $6e3edeb6$

Elementary SHA operation



[处理冲突的常用方法]

[1. 开放定址法 (空缺编址法)]

$$H_i = (H(\text{key}) + d_i) \text{ MOD } m \quad 0 \quad H(\text{key}) \quad m-1$$

$$i=1, 2, \dots, k \quad (k \leq m-1)$$



m: 哈希表的表长; d_i : 增量序列

1) 线性探测再散列

$$d_i = 1, 2, \dots, m-1$$

缺陷: 有聚集(堆积)现象—非同义词地址冲突。

e.g: 假定采用的 hashing 函数为： $H(\text{key}) = \text{key} \text{ MOD } 11$ 假定的关键字序列为： 17、60、29、38 ； 它们的散列过程为：

$H(17) = 6$

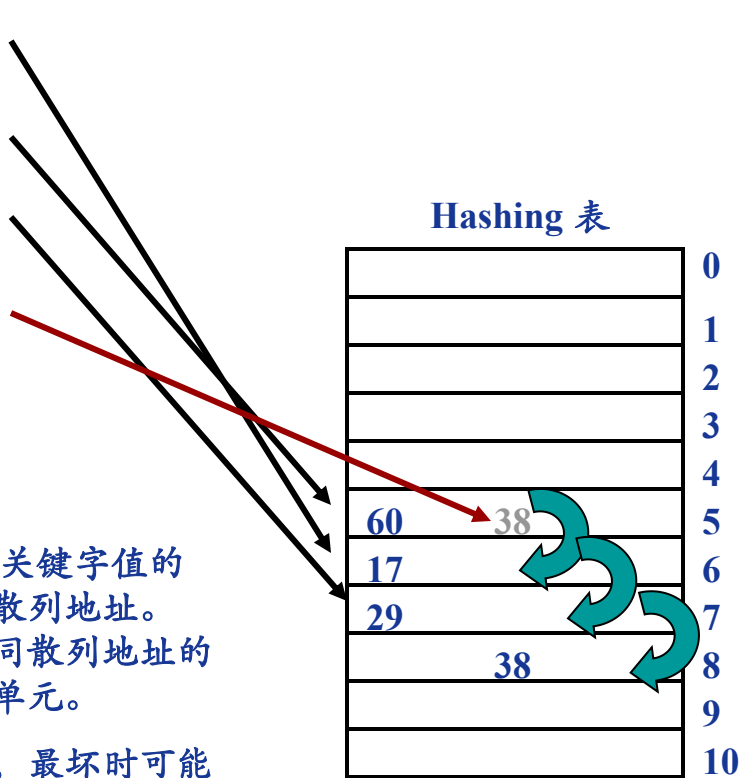
$H(60) = 5$

$H(29) = 7$

$H(38) = 5$

冲突：

初级冲突：不同关键字值的结点得到同一个散列地址。
二次聚集：同不同散列地址的结点争夺同一个单元。
结果：冲突加剧，最坏时可能达到 $O(n)$ 级代价。



当散列 38 时发生冲突，
同 60 争夺第 5 个单元

解决办法：探测下一个
空单元

$H_i = (H(\text{key}) + d_i) \text{ MOD } 11$

思考：假定有k个关键字互为同义词，若用线性探测再散列法把这k个关键字存入散列表中，至少要进行多少次探测？

$\frac{k(k+1)}{2}$
BUPT SCST

[线性探测再散列-查找算法]

```
ElemType *Find(int k, ElemType ht[], int m)//ht为哈希表, m为其长度
{
    int homeBucket, currentBucket;
    homeBucket = h(k);//h为哈希函数
    for (currentBucket = homeBucket;
        ht[currentBucket] && ht[currentBucket].key != k; )
    {
        currentBucket = (currentBucket + 1) % m;//线性探测再散列
        if (currentBucket == homeBucket)
            return NULL;//回到起点, 不存在关键字为k的记录, 查找失败;
    }
    if (ht[currentBucket]->key == k)
        return ht[currentBucket];
    return NULL;//找到空槽位, 查找失败
}
```

[线性探测再散列-插入算法]

void Insert(ElemType data, ElemType &ht[], int m)//ht为哈希表, m为其长度

```
{
    KeyType k = data.key;
    int homeBucket, currentBucket;
    homeBucket = h(k); //h为哈希函数
    currentBucket = homeBucket;
    while( ht[currentBucket] ) //ht[currentBucket]不为空
    {
        currentBucket = (currentBucket + 1)%m; //线性探测再散列
        if(currentBucket == homeBucket)
            FatalError("Space is Full!!"); //空间满, 退出
    }
    ht[currentBucket] = data;
}
```

2) 二次探测再散列

$$d_i = 1^2, -1^2, 2^2, -2^2, 3^2, \dots, \pm k^2$$

$$k \leq m/2$$

缺陷：不易探查整个散列空间。

3) 伪随机探测再散列

$$d_i = \text{伪随机数序列}$$

4) 双重散列函数探查法

$$d_i = i * h_1(\text{key}) \quad 0 < i \leq m-1$$

要求： $h_1(\text{key})$ 的值与 m 互素。

m 为素数时， $h_1(\text{key})$ 可取1到 $m-1$ 之间的任何数，
建议：

$$h_1(\text{key}) = \text{key} \bmod (m-2) + 1$$

m 为2的方幂时， $h_1(\text{key})$ 可取1到 $m-1$ 之间的任何奇数

注意：开放定址法不宜执行删除操作

[2. 再哈希法]

$$H_i = RH_i(\text{key}) \quad i=1,2, \dots, k$$

出现冲突时，采用多个 hashing 函数计算散列地址，直到找到空单元为止。

如： $H_1(\text{key}) = \text{key} \text{ MOD } 11$

$$H_2(\text{key}) = (\text{key} + \text{MOD } 9) + 1$$

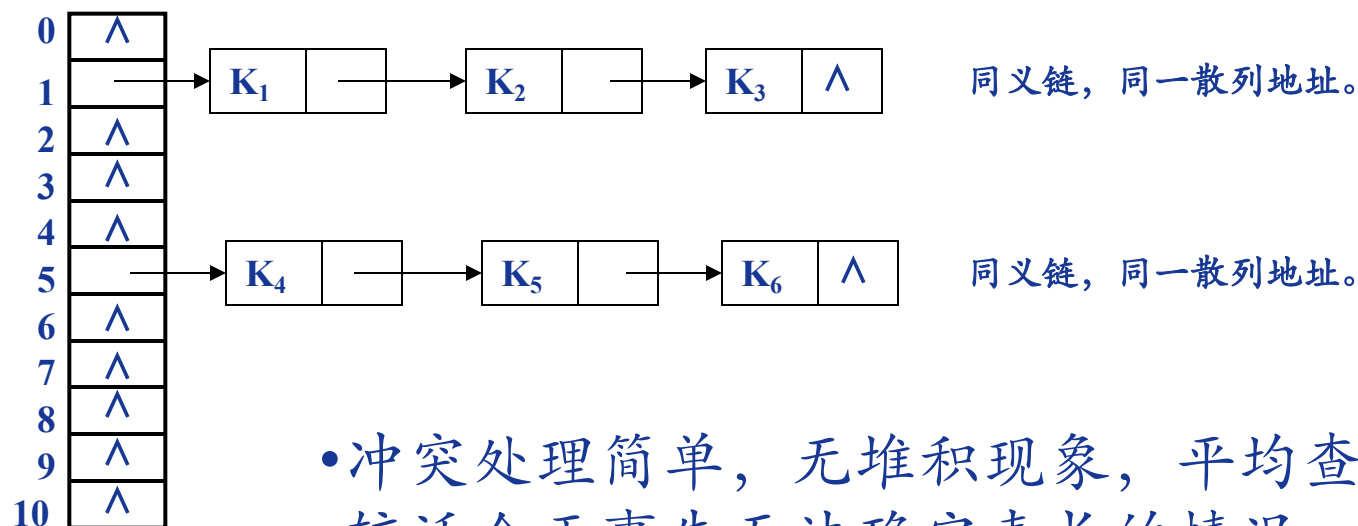
$$H_3(\text{key}) = \dots\dots\dots$$

.....

$$H_{10}(\text{key}) = \dots\dots\dots$$

[3. 链地址法]

为每个哈希地址建立一个单链表，存储所有具有同义词的记录。



- 冲突处理简单，无堆积现象，平均查找长度较短；
- 较适合于事先无法确定表长的情况；
- 可取 $\alpha \geq 1$ ，当结点信息规模较大时，节省空间
- 删除结点的操作易于实现

【链地址法数据结构定义】

```
typedef struct ListNode *Position;
typedef Position List;
struct ListNode {
    ElemType data;
    Position next;
};
typedef struct HashTbl *HashTable ;
Struct HashTbl {
    int TableSize;
    List * thelists; //数组，初始化时分配空间
};
HashTable InitializeTable(int TableSize); //初始化链地址法哈希表
void DestroyTable(HashTable H); //销毁链地址法哈希表
Position Find(ElemType key, HashTable H); //查找
void Insert(ElemType key, HashTable &H); //插入
ElemType Retrieve(Position P); //获取关键字
```

[链地址法-初始化算法]

HashTable InitializeTable(int TableSize)

```
{
    HashTable H; int i;
    if (TableSize < MinTableSize) return NULL; //表太小，退出
    H = malloc(sizeof( struct HashTbl)); //分配一个表结构
    if (H==NULL) FatalError(“Out of Space!!”); //分配失败退出
    H->TableSize = TableSize;
    H->TheLists = malloc(sizeof(list)* H->TableSize ); //分配指针数组空间
    if (H->TheLists == NULL) FatalError(“Out of Space!!”);
    for (i = 0; i < H->TableSize; i++) //可选项，为所有单链表分配头结点
    {
        H->TheLists[i] = malloc(sizeof(struct ListNode));
        if (H->TheLists[i] ==NULL) FatalError(“Out of Space!!”);
        else H->TheLists[i]->next = NULL;
    }
    return H;
}
```

[查找算法](假设哈希函数为**h**)

Position Find(KeyType key, HashTable H)

```
{
    Position P; List L;
    L = H->TheLists[h(key)];
    P = L->next;
    while (P!=NULL &&
           P->data.key != key)
        P = P->next;
    return P;
}
```

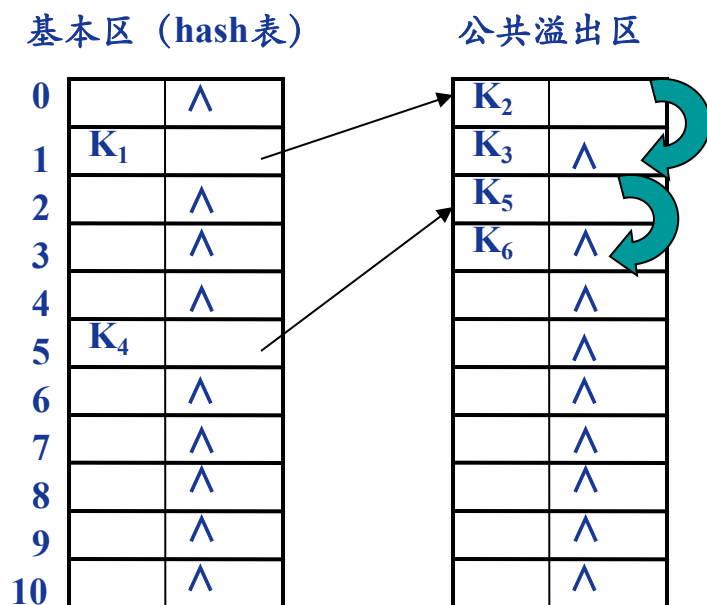
[插入算法]

Void Insert(ElemType data, HashTable &H)

```
{
    position Pos, New;
    Pos = Find(data.key ,H);
    if (Pos == NULL)
    {
        New = malloc(sizeof(struct ListNode))
        if(New == NULL) FataError("out of space!");
        else//头插法
        {
            L = H->TheLists[h(data.key)];
            New->next = L->next;
            New->data = data;
            L->next = New;
        }
    }
}
```

[4. 建立公共溢出区]

将发生冲突的结点都存放在一个公共溢出区内。通常用于组织存在于外存设备上的数据文件。hash表(基本区)只存放一个记录。发生冲突的记录都存放在公共溢出区内。



[哈希表的查找及其分析]

查找方法同 hashing 函数的产生办法。

例：key = en to tre fire fem seks syv

$H(\text{key}) = 2 \quad 0 \quad 3 \quad 0 \quad 4 \quad 8 \quad 1$

解决冲突方法：线性探测再散列

0	1	2	3	4	5	6	7	8
to	fire	en	tre	fem	syv			seks

$$ASL_s = (1+1+1+2+1+1+5)/7 = 12/7$$

$$ASL_f = (7+6+5+4+3+2+1+1+8)/9 = 37/9$$

$$\alpha = 7/9$$

[平均查找长度与哈希表的装填因子的关系]

$$\alpha = \frac{\text{表中填入的记录数}}{\text{哈希表的长度}}$$

在一般情况下， $\alpha \downarrow$ ，冲突的可能性 $\downarrow$ ，ASL $\downarrow$ ，但空间的浪费 $\uparrow$ 。

	成功ASL	失败ASL
线性探测再散列	$\frac{1}{2}(1 + \frac{1}{1-\alpha})$	$\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$
伪随机探测再散列	$-\frac{1}{\alpha} \ln(1-\alpha)$	$\frac{1}{1-\alpha}$
链地址法	$1 + \frac{\alpha}{2}$	$\alpha + e^{-\alpha}$

作业

BUPT

1. 包括 n 个关键码的 m 阶B-树在一次检索中最多涉及多少个结点？（要求写出推导过程。）
2. 试从空树开始，画出按以下次序向3阶B-树中插入关键码的建树过程：20，30，50，52，60，68，70。如果此后删除50和68，画出每一步执行后B-树的状态。
3. 已知二叉树排序树中某结点指针 p ，其双亲结点指针为 fp ， p 为 fp 的左孩子，试编写算法，删除 p 所指结点。

```
typedef struct node
{int data; struct node *left, *right;
}BiTNode, *BiTree;

void Delete(BiTree &root, BiTree p, BiTree fp);
```

4. 试画出从空树开始，由字符序列(t,d,e,s,u,g,b,j,a,k,r,i)构成的二叉平衡树，并为每一次的平衡处理指明旋转类型。

5. 已知含12个关键字的有序表及其相应权值为：

1 2 3 4 5 6 7 8 9 10 11 12

关键字 A B C D E F G H I J K L

权值 4 6 3 4 9 3 2 6 1 5 3 4

(1)画出对以上有序表进行折半查找的判定树，求折半查找时查找成功的平均查找长度ASL。

(2)若为等概率查找，求折半查找时查找成功的平均查找长度ASL。

6. 假设一棵平衡二叉树的每个结点都标明了平衡因子bf,试设计一个算法，利用bf值求平衡二叉树的高度。 **int Height (BSTree t)**

typedef struct node

{int data; struct node *left, *right;

}BiTNode, *BSTree;

7. 选取哈希函数 $H(k)=(3k) \text{ MOD } 11$ 。

- 1) 用线性探测开放定址法处理冲突,
- 2) 用链地址法处理冲突

试在0~10的散列地址空间中对关键字序列(22, 41, 53, 46, 30, 13, 01, 67)造哈希表, 并求等概率情况下查找成功时的平均查找长度和不成功时的平均查找长度以及装填因子 α 。